

Armstrong Number: $n = \underline{\underline{153}} \Rightarrow \underline{1^3 + 5^3 + 3^3} = n \rightarrow \begin{matrix} T \\ \hookrightarrow F \end{matrix}$

power = 0

temp = temp + pow(d, 3)

= 27 + p(5, 3)

= 27 + 125

temp = 153

if (n == temp)

 ↳ true

 ↳ false

$n \% 10$

$d = \underline{\underline{153}} \% 10 = 3$

$n = n / 10$

$153 / 10 = 15$

while (n > 0)

{

 power++

}

$$\begin{aligned} \text{power} = 0 \quad & \text{temp} = \text{temp} + \text{pow}(d, 3) \\ & = 27 + \text{p}(5, 3) \\ & = 27 + 125 \end{aligned}$$

temp = 153

```
if (n == temp)
    ↳ true
    ↳ false
```

$$d = 153 \cdot \frac{1}{10} = 3$$
$$\eta = \eta / 10$$
$$153 / 10 = 15$$
$$\text{While}(n > 0)^{1/10} = 0$$

power#;

2

Encapsulation : $\rightarrow$

Encapsulation $\rightarrow$ The process of wrapping the data members of a class inside a block of code so that they cannot be accidentally modified, is called encapsulation. It is done using the private access modifier.

To provide access outside class the class, we use special methods called "getters" & "setters".

The "getters" & "setters" should be public.

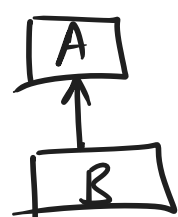
* Inheritance: The property of inheriting / accessing the data members of the Parent Class into the respective child classes, is called inheritance. Here, the child classes can also have properties of their own & methods of their own.

Types of Inheritance :

- ① Single
- ② Multi-level
- ③ Multiple
- ④ Hierarchical
- ⑤ Hybrid

Inheritance

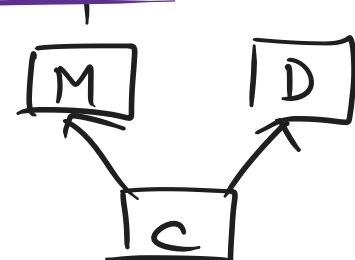
Single Level :



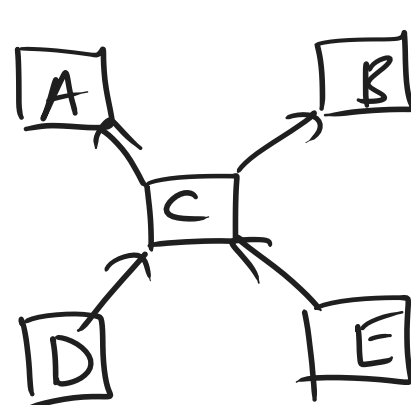
Multi-level :



Multiple:

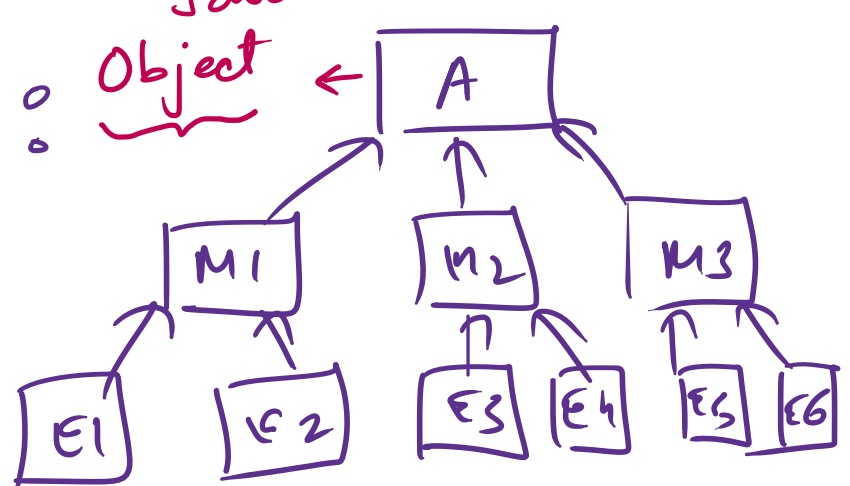


Hybrid:



Hierarchical

Java
Object



Polymorphism :

Latin
 poly ← morph
 many forms/shapes

Anvitha is not changing. Her role is changing.

→ Classroom	:	Student
→ Home	:	Daughter / Sister
→ Restaurant	:	Customer
→ Playground	:	Athlete / Player

Types of Polymorphism:

- ① Static Polymorphism : Compile Time : Overloading →
[Same Class]

- (11) Dynamic Polymorphism : Runtime : Overriding →
[Multiple Classes]